

Intelligent Edge-Computing Mirror

1. Introduction

The Smart AI Mirror is a sophisticated, IoT display that seamlessly integrates artificial intelligence, computer vision, and voice processing into an everyday object. Designed to operate behind a two-way acrylic mirror, the system utilizes a vertically inclined monitor to present a floating, customized digital interface as your daily need object.

By utilizing a Raspberry Pi as a local edge device, the mirror processes sensory inputs—like continuous camera feeds, motion detection and audio—directly on the device. This local processing ensures low latency and user privacy, while asynchronous cloud API connections provide dynamic data such as weather, news, advanced conversational AI, and Spotify music control. The result is a highly responsive, multi-user smart assistant that responds to both natural language and physical hand gestures.

2. Project Requirements

Functional Requirements

- **Multi-User Authentication:** The system must use facial recognition to identify the current user and load their personalized widget layout.
- **Gesture-Based Control:** The system must track hand movements to control an on-screen cursor and recognize a clenched fist to 'grab' and move widgets (e.g., calendar, to-do list).
- **Conversational AI Assistant:** The system must listen for an offline wake word ('Hey Mirror'), process human questions, and provide audio text-to-speech responses alongside visual updates.
- **API Integrations:** The system must connect to external services to fetch real-time weather and news, and authenticate with Spotify for music playback control.
- **Smart SOS Feature:** The system must recognize a specific emergency trigger (voice or gesture) to instantly send an SMS via an SMS module.
- **Automated Power Management:** The system must use a motion sensor to detect human presence, turning the display on when a user is nearby and putting it to sleep when the room is empty.

Non-Functional Requirements

- **Low Latency:** Gesture-to-cursor process must update with minimal delay, mimicking the responsiveness of a physical mouse.
- **Concurrency:** Camera feed, microphone listening loop, and UI must run simultaneously without blocking during heavy API calls.
- **UI Transparency:** The UI must render transparently through the two-way acrylic glass.

3. Implementation Strategy

Area 1: Face Detection & Gesture Control

This part handles visual inputs. Using Python, OpenCV, and MediaPipe, the camera feed is processed in the background. It maps palm coordinates to screen resolution and calculates fingertip distance to determine grab states. It also manages the PIR motion sensor to control the monitor's power state along with facial recognition and authentication feature.

Area 2: AI, Voice, Audio & APIs

This module handles auditory inputs and external API communication. A Python program for wake-word detection and routes commands to an LLM for responses. It fetches external API data (Spotify, Weather) and triggers Smart SOS emergency protocols.

Area 3: UI Building

This module manages the visual interface. Built with HTML, CSS, and JavaScript, it runs in a kiosk-mode Chromium browser. It listens to a WebSocket server for data from the Vision and AI modules, updating cursor positions, widgets, and user profiles dynamically.

4. Required Components

Hardware Components

- Processing Unit: Raspberry Pi.
- Display: LCD monitor.
- Frame & Glass: Custom wooden or 3D-printed frame with two-way acrylic mirror sheet.
- Vision Input: Raspberry Pi Camera Module.
- Audio Input/Output: USB Microphone and external speakers.
- Sensors: PIR Motion Sensor for automated power management.
- Actuator: GSM Module for SMS calibration.

Software & Frameworks

- Backend Core: Python.
- Machine Learning & Vision: OpenCV and Google MediaPipe.
- Audio Processing: Picovoice Porcupine, OpenAI API or Gemini API, SpeechRecognition library.
- Frontend UI: HTML, CSS, JavaScript.
- Communication Layer: WebSockets.

5. Summary

The Smart AI Mirror demonstrates the integration of hardware engineering, local machine learning, and reactive web design. By separating the system into Vision, Voice, and UI modules, development can occur in parallel before deployment to the Raspberry Pi.

WebSockets bridge continuous Python processes with a lightweight JavaScript frontend, ensuring the final system remains responsive, scalable, and highly interactive.

